

MEMORIA DE PRÁCTICAS

2º Ordenación
Rápida

Pablo Fazio Arrabal
2º Doble Grado Ing. Informática y
Matemáticas

ORDENACIÓN RÁPIDA

En esta práctica, el alumno estudiará y desarrollará diversos algoritmos de ordenación en Java para ordenar ascendentemente un vector de valores.

a) En la clase OrdenacionRapida implementar el método ordRapidaRec que tome como parámetros un vector v y dos enteros, izq y der, y ordene los elementos de dicho vector desde izq a der (los restantes elementos deberán quedar como en el vector original). Tal como se ha definido la cabecera del método, será posible ordenar vectores con cualquier tipo de elementos, siempre que dichos elementos implementen el interfaz Comparable; es decir, siempre que definan el método compareTo. En la cabecera, T es un tipo genérico que denota el tipo de los elementos del vector. Para poder comparar elementos de tipo T el alumno tendrá que usar el método compareTo, que se invoca sobre un objeto y que toma como parámetro otro objeto, devolviendo 0 si ambos son iguales, menor que 0 si el objeto sobre el que se invoca es menor al objeto parámetro, o mayor que 0 en caso contrario.

b) Implementar una variante del método de ordenación rápida, en la clase OrdenacionRapidaBarajada, de manera que previamente a la ordenación se mezclen aleatoriamente los elementos del vector. Comprobar ahora el funcionamiento de esta variante del método.

c) Aprovechando el método partir implementado en el primer apartado, completar en la clase BuscaElem la implementación eficiente del método kesimoRec que tome como parámetros un vector v (desordenado) y tres enteros, y encuentre el elemento que debería estar en la posición k (si este estuviera ordenado, pero sin ordenar el vector) de dicho vector desde izq a der.

Durante esta práctica, acometeremos tres implementaciones claves para la resolución de la ordenación de diversas maneras usando métodos predefinidos y métodos construidos en anteriores apartados. Los archivos auxiliares de la práctica son bastantes extensos por lo que simplemente manejaremos las clases OrdenacionRapida, OrdenacionRapidaBarajada y BuscaElem, aquellas que debemos entregar, que serán herederas de una clase abstracta Ordenacion. Aunque también comentaremos brevemente las clases de prueba TestsCorreccion y TestsTiempos que nos ayudarán a probar nuestros ficheros y a dar una visión comparativa sobre otros métodos de ordenación dados.

Veamos cada apartado por separado, pudiendo definir una relación entre ellos cuando la ocasión lo requiera. En primer lugar, veamos la clase OrdenacionRapida.java, es claro que debemos definir un método auténticamente recursivo para que nuestra función rebaje su orden de complejidad básico mediante el uso de un pivote gracias a la implementación QuickSort, vista en clase, dicho elemento pivote lo encontraremos gracias a un método *partir*, que reorganizará los valores iniciales.

Veamos por primera vez nuestra clase OrdenacionRapida.java:

```

public class OrdenacionRapida extends Ordenacion {

    public static <T extends Comparable<? super T>> void ordenar(T v[]) {
        ordRapidaRec(v, 0, v.length - 1);
    }

    public static <T extends Comparable<? super T>> void ordRapidaRec(T
v[], int izq, int der) {

        if (izq < der) {
            int medio = partir(v, v[izq], izq, der);

            ordRapidaRec(v, izq, medio - 1);
            ordRapidaRec(v, medio + 1, der);
        }

    }

    public static <T extends Comparable<? super T>> int partir(T v[], T
pivote, int izq, int der) {

        int i = izq+1;
        int j = der;

        do {
            while((i<=j) && (v[i].compareTo(pivote)<=0)){
                i++;
            }
            while((i<=j) && (v[j].compareTo(pivote)>0)){
                j--;
            }
            if (i<j) {
                intercambiar(v,i,j);
            }
        }while (i < j);

        intercambiar(v,izq,j);

        return j;
    }

    public static void main(String args[]) {

        // Un vector de enteros
        Integer vEnt[] = { 3, 8, 6, 5, 2, 9, 1, 1, 4 };
        ordenar(vEnt);
        System.out.println(vectorAsString(vEnt));

        // Un vector de caracteres
        Character vCar[] = { 'd', 'c', 'v', 'b' };
        ordenar(vCar);
        System.out.println(vectorAsString(vCar));

    }
}

```

Veamos paso a paso, lo que hemos realizado en esta clase:

```
public static <T extends Comparable<? super T>> void ordenar(T v[]) {
    ordRapidaRec(v, 0, v.length - 1);
}
```

En primer lugar, este método será el que se llame desde nuestra prueba y llamará a su vez al método implementado *ordRapidaRec* que tomará el vector dado e inicializará en la primera posición y la última del vector.

```
public static <T extends Comparable<? super T>> void ordRapidaRec(T v[], int
    izq, int der) {
    if (izq < der) {
        int medio = partir(v, v[izq], izq, der);

        ordRapidaRec(v, izq, medio - 1);
        ordRapidaRec(v, medio + 1, der);
    }
}
```

El método recursivo *ordRapidaRec* seguirá las pautas básicas del algoritmo *QuickSort*, en primer lugar vemos que están bien definidos los datos de entrada, luego, mediante el método *partir* asignaremos el valor medio del vector que usaremos para separar en dos la llamada recursiva.

```
public static <T extends Comparable<? super T>> int partir(T v[], T pivote,
    int izq, int der) {
    int i = izq+1; int j = der;

    do {
        while((i<=j) && (v[i].compareTo(pivote)<=0)){
            i++;
        }
        while((i<=j) && (v[j].compareTo(pivote)>0)){
            j--;
        }
        if (i<j) {
            intercambiar(v,i,j);
        }
    }while (i < j);

    intercambiar(v,izq,j);

    return j;
}
```

Por último, veamos que hace el método *partir* donde realizamos un bucle en el cual cambiamos de posición a la izquierda aquellos elementos que sean menores que el pivote y a la derecha aquellos mayores (el caso de la igualdad es trivial), ayudados gracias al método *intercambiar* implementado en la clase *Ordenacion*.

Además creamos una clase prueba para ver que funciona nuestra clase en elementos de tipo `Integer` y `Character` que implementan la clase `Comparable`. Esta clase será muy útil para las dos posteriores, gracias a sus métodos.

Veamos ahora la clase `OrdenacionRapidaBarajada.java`:

```
public class OrdenacionRapidaBarajada extends OrdenacionRapida {

    // Implementación de QuickSort con reordenación aleatoria inicial
    (para comparar tiempos experimentalmente)

    public static <T extends Comparable<? super T>> void ordenar(T v[]) {
        barajar(v);
        OrdenacionRapida.ordenar(v);
    }

    // reordena aleatoriamente los datos de un vector

    private static <T> void barajar(T v[]) {

        for (int i = v.length - 1; i >= 0; i--) {
            int random = aleat.nextInt(i + 1);
            intercambiar(v, random, i);
        }

    }

}
```

En este caso simplemente será una clase heredera de `OrdenacionRapida` en la cual antes de comenzar a ordenar el array dado usaremos un método `barajar` para realizar una reordenación aleatoria de los elementos mediante un generador.

```
private static <T> void barajar(T v[]) {

    for (int i = v.length - 1; i >= 0; i--) {
        int random = aleat.nextInt(i + 1);
        intercambiar(v, random, i);
    }

}
```

Realizaremos un bucle en este método donde iremos cambiando los elementos del vector mediante un elemento `Random aleat` desde `Ordenacion`. Es claro el funcionamiento, tomamos todos los elementos en la primera iteración y poco a poco restringimos los casos que intercambiamos.

Una vez terminado el método, simplemente invocaremos al método principal creado en la clase `OrdenacionRapida`.

Por último tomemos la clase `BuscaElem.java`:

```

public final class BuscaElem {

    public static <T extends Comparable<? super T>> T kesimo(T v[], int k)
    {
        return kesimoRec(v, 0, v.length - 1, k);
    }

    public static <T extends Comparable<? super T>> T kesimoRec(T v[], int
    izq, int der, int k) {
        int indiceRes = OrdenacionRapida.partir(v, v[izq], izq, der);
        if (k == indiceRes) {
            return v[k];
        } else if (k < indiceRes) {
            return kesimoRec(v, izq, indiceRes - 1, k);
        } else {
            return kesimoRec(v, indiceRes + 1, der, k);
        }
    }

    public static void main(String[] args) {
        int maxvector;
        int i, k;

        Scanner sc = new Scanner(System.in);
        System.out.print("Introduce el número de posiciones del vector:
    ");
        maxvector = sc.nextInt();
        Integer v[] = new Integer[maxvector];

        System.out.print("Introduce " + maxvector + " enteros separados
    por espacios: ");
        for (i = 0; i < maxvector; i++)
            v[i] = sc.nextInt();
        System.out.print("Introduce la posición k deseada (de 1-" +
    maxvector + "): ");
        k = sc.nextInt();
        Integer elem = kesimo(v, k - 1);
        System.out.print("El elemento " + k + "-esimo es: " + elem);

        sc.close();
    }
}

```

Para comenzar, este método será el que se llame desde nuestra prueba y llamará a su vez al método implementado *kesimoRec* que tomará el vector dado e inicializará en la primera posición y la última del vector, así como la posición deseada para la que buscar.

```

public static <T extends Comparable<? super T>> T kesimo(T v[], int k)
{
    return kesimoRec(v, 0, v.length - 1, k);
}

```

Después usando el método *partir* desde *OrdenacionRapida* debemos ir haciendo llamadas recursivas de la función hasta que la posición dada por el usuario coincida con el resultado del método donde devolveremos el valor de la posición del array ordenado.

```
public static <T extends Comparable<? super T>> T kesimoRec(T v[], int izq,
int der, int k) {
    int indiceRes = OrdenacionRapida.partir(v, v[izq], izq, der);
    if (k == indiceRes) {
        return v[k];
    } else if (k < indiceRes) {
        return kesimoRec(v, izq, indiceRes - 1, k);
    } else {
        return kesimoRec(v, indiceRes + 1, der, k);
    }
}
```

Ya por último usaremos un método main para comprobar el funcionamiento de la clase.